CS-501: Machine Learning Foundations

Comprehensive Academic Lecture Notes & Study Guide
Department of Computer Science & Engineering | Semester Fall

Chapter 1: Supervised Learning & Foundations

1.1 Definition and Objective

Supervised learning is the machine learning task of learning a function that maps an input to an output based on example input-output pairs. The training dataset consists of N pairs: D = {(x_1, y_1), (x_2, y_2), ..., (x_N, y_N)}, where each x_i is a feature vector in R^d, and y_i is the target ground truth label.

When y_i is continuous, the problem is termed **Regression** (e.g., predicting housing prices). When y_i is discrete and categorical, the problem is termed **Classification** (e.g., medical diagnosis, spam filtering).

1.2 Linear Regression Formulation

In linear regression, the hypothesis model assumes a linear relationship between features and target:

```
Hypothesis: h_theta(x) = theta_0 + theta_1*x_1 + ... + theta_d*x_d = theta^T * x
```

The primary goal is to find the parameter vector theta that minimizes the discrepancy between predictions h_theta(x) and actual labels y.

Chapter 2: Loss Functions & Optimization

2.1 Mean Squared Error (MSE)

For continuous regression targets, the standard objective function is the Mean Squared Error (MSE) cost function:

```
J(theta) = (1 / (2 * N)) * sum_{i=1}^N (h_theta(x_i) - y_i)^2
```

The factor of 1/2 is mathematically convenient because it cancels out when taking the partial derivative with respect to theta.

2.2 Cross-Entropy Loss for Classification

For binary classification where y in {0, 1}, we use the Binary Cross-Entropy (Log Loss) function derived from maximum likelihood estimation:

```
L(theta) = - (1/N) * sum_{i=1}^N [ y_i * log(p_i) + (1 - y_i) * log(1 - p_i) ]
```

where p_i = sigmoid(theta^T * x_i). This penalty grows exponentially as predicted probability diverges from actual label.

2.3 Gradient Descent Algorithm

Gradient Descent is an iterative first-order optimization algorithm. At each step, parameters are updated in the direction of steepest descent:

```
Update Rule: theta := theta - alpha * grad(J(theta))
```

Here, **alpha** denotes the **learning rate**. If alpha is too small, convergence is painfully slow. If alpha is too large, gradient descent can overshoot the minimum and diverge.

Chapter 3: Overfitting, Bias-Variance, & Regularization

3.1 The Bias-Variance Tradeoff

Total expected prediction error decomposes into three fundamental components: Bias^2 + Variance + Irreducible Error. **High Bias (Underfitting)** occurs when a model is overly simplistic and cannot capture the underlying structure of the data. **High Variance (Overfitting)** occurs when a model fits the training data noise rather than general patterns, failing on test data.

3.2 L2 Regularization (Ridge)

Ridge regression penalizes large weights by appending the squared L2 norm to the cost function:

```
J_ridge(theta) = J(theta) + (lambda / (2 * N)) * sum_{j=1}^d theta_j^2
```

L2 regularization shrinks coefficients smoothly toward zero but rarely makes them exactly zero.

3.3 L1 Regularization (Lasso)

Lasso regression penalizes the sum of absolute values of parameters:

```
J_lasso(theta) = J(theta) + (lambda / N) * sum_{j=1}^d |theta_j|
```

L1 regularization drives irrelevant feature weights exactly to zero, functioning as an automatic **feature selection** technique.

Chapter 4: Evaluation Metrics & Deep Learning

4.1 Classification Metrics: Precision, Recall, and F1

Accuracy alone is misleading on imbalanced datasets. We rely on the Confusion Matrix comprising True Positives (TP), False Positives (FP), True Negatives (TN), and False Negatives (FN).

Precision: TP / (TP + FP) — Fraction of relevant instances among retrieved instances. Measures quality of positive predictions.
Recall (Sensitivity): TP / (TP + FN) — Fraction of actual positives that were correctly identified.
F1 Score: 2 * (Precision * Recall) / (Precision + Recall) — The harmonic mean of Precision and Recall.

4.2 Neural Network Activation Functions

Activation functions introduce non-linearity, allowing networks to learn complex decision boundaries:
• **Sigmoid:** sigma(z) = 1 / (1 + exp(-z)). Maps to (0, 1). Susceptible to vanishing gradient problem in deep layers.
• **ReLU (Rectified Linear Unit):** f(z) = max(0, z). Most popular in hidden layers; computationally efficient and mitigates vanishing gradients.
• **Softmax:** Normalizes logits into a valid probability distribution over K mutually exclusive classes: P(y=k) = exp(z_k) / sum_j exp(z_j).